

Guia Detalhado de Implementação: Cliente DNS MX em C

Fundamentos de Redes de Computadores • Trabalho 01 • 2026.2

Este guia complementa o "Guia de Implementação" original. Ele não traz código-fonte. Explica o que cada parte do programa deve fazer, em que ordem construí-la, quais decisões tomar, quais erros são comuns e como confirmar que cada etapa está certa antes de passar para a seguinte.

Como usar este guia

- Siga as fases na ordem. Cada fase termina com um **Checkpoint**, que é um teste objetivo. Só avance quando o checkpoint passar.
- Cada fase tem as seções **Objetivo**, **O que fazer**, **Decisões e cuidados**, **Erros comuns** e **Checkpoint**.
- Termos como "cursor" e "offset" sempre significam posição em bytes a partir do início da mensagem DNS (byte 0).
- O programa final cabe em um único arquivo, `meu_cliente.c`, mais o `README.md`.

Parte A. Visão geral

A.1 O que o programa faz

1. Recebe dois argumentos: o domínio e o IP do servidor DNS.
2. Monta, byte a byte, uma mensagem DNS de consulta do tipo MX, classe IN, com recursão desejada.
3. Envia essa mensagem por UDP à porta 53 do servidor informado.
4. Espera a resposta por até 2 segundos, com no máximo 3 envios.
5. Valida a resposta recebida e extrai os registros MX.
6. Imprime o resultado no formato exigido, ou uma das três mensagens de falha.

A.2 O que o programa NÃO faz (escopo)

- Não usa biblioteca DNS (`res_query`, `c-ares`, `dnspython`) nem chama `dig`.
- Não usa `getaddrinfo` para resolver o domínio consultado. O IP do servidor já vem como literal.
- Não faz fallback para TCP, não usa EDNS, não valida DNSSEC e não tem cache.
- Não percorre servidores raiz: pede recursão ao servidor indicado.
- Não tem interface gráfica, threads, menu ou `scanf`.

A.3 Arquitetura em camadas

Pense no programa como cinco camadas, cada uma dependendo apenas das de baixo:

Camada	Responsabilidade	Depende de
1. Utilitários de bytes	Ler e escrever inteiros de 16 bits em ordem de rede	nada
2. Codificação	Transformar o nome textual em labels DNS	camada 1
3. Mensagem de consulta	Montar cabeçalho e pergunta	camadas 1 e 2
4. Interpretação	Ler nomes comprimidos, validar e classificar a resposta	camada 1
5. Transporte e <code>main</code>	Socket, prazos, retransmissão, argumentos, saída	todas

Regra de ouro: as camadas 1 a 4 não tocam em rede nem em `stdout`. Isso permite testá-las com bytes fabricados à mão.

A.4 Fluxo completo

1. `main` valida a quantidade de argumentos.
2. O domínio é normalizado e codificado. Se inválido, mensagem de erro de uso em `stderr` e saída com falha, sem enviar nada.
3. O IP do servidor é convertido. Se inválido, erro em `stderr`.
4. O ID aleatório é lido de `/dev/urandom`.
5. A consulta é montada e o socket UDP é criado.
6. O ciclo de envio e espera roda no máximo 3 vezes.

7. Uma resposta válida é classificada em: sucesso, domínio inexistente, sem MX ou falha de coleta.
8. O resultado é impresso, o socket é fechado e o programa retorna o código de saída.

Parte B. Fases de implementação

Fase 0. Ambiente e esqueleto

Objetivo

Ter um projeto que compila sem avisos e uma estrutura de arquivos pronta.

O que fazer

1. Crie uma pasta de trabalho com `meu_cliente.c` e `README.md`. O repositório atual já tem o esqueleto de `main` que valida `argc == 3`.
2. Defina a linha de compilação padrão e use-a sempre: padrão C11, macro `_POSIX_C_SOURCE=200809L`, avisos `-Wall -Wextra -Wpedantic` e otimização `-O2`.
3. Liste os cabeçalhos necessários agrupados por função:
 - tipos e tamanhos: `stdint.h`, `stddef.h`
 - texto e memória: `stdio.h`, `string.h`, `stdlib.h`, `ctype.h`
 - rede: `sys/socket.h`, `netinet/in.h`, `arpa/inet.h`
 - tempo e espera: `poll.h`, `time.h`, `errno.h`
 - recursos: `unistd.h`
4. Comente no topo do arquivo a lista de funções planejadas (o esqueleto atual já faz isso). Isso serve de índice.
5. Crie as constantes do protocolo em um só lugar, com nomes claros: tamanho do cabeçalho (12), tipo MX (15), classe IN (1), porta (53), prazo por tentativa (2000 ms), número de tentativas (3), limite de saltos de CNAME (8), tamanho máximo de nome (255), tamanho máximo de label (63), tamanho do buffer de consulta (512) e de resposta (65535).
6. Defina um `enum` de resultados: sucesso, sem MX, NXDOMAIN, resposta irrelevante (ignorar) e erro/inconclusivo.

Decisões e cuidados

- **Desenvolvimento no macOS:** o compilador chamado `gcc` no Mac costuma ser o `clang`. Isso funciona para desenvolver, mas a verificação final de compilação deve ser feita em Linux (laboratório, WSL ou máquina virtual), como o guia original pede. Registre no README o ambiente de fato usado.
- Evite variáveis globais para tamanhos e buffers. Toda função de protocolo recebe o buffer e o seu tamanho como parâmetros.
- Prefira inteiros de tamanho explícito (`uint8_t`, `uint16_t`) para dados de protocolo e `size_t` para índices.

Erros comuns

- Esquecer a macro POSIX e obter erros de declaração de `clock_gettime` ou `struct sockaddr_storage`.
- Misturar `int` com `size_t` em comparações, gerando avisos de sinal.

Checkpoint

O programa compila sem nenhum aviso e, ao rodar com argumentos errados, imprime a mensagem de uso em `stderr` e retorna 1.

Fase 1. Utilitários de ordem de rede

Objetivo

Ler e escrever números de 16 bits sem depender da arquitetura da máquina.

O que fazer

1. Crie uma função que escreve um valor de 16 bits em dois bytes consecutivos, byte mais significativo primeiro (big-endian).
2. Crie a função inversa, que lê dois bytes e devolve o valor de 16 bits.
3. Use sempre essas duas funções para qualquer campo do cabeçalho DNS, nunca um cast de ponteiro nem uma `struct` copiada para o buffer.

Decisões e cuidados

- O motivo de evitar `struct`: o compilador pode inserir preenchimento (padding) e a ordem dos bytes depende da CPU. Escrever campo a campo elimina os dois problemas.
- A função de leitura não verifica limites. **Quem chama** deve garantir que há pelo menos 2 bytes disponíveis. Documente essa pré-condição em comentário.

Erros comuns

- Fazer o deslocamento sem converter para `uint8_t` e escrever valores maiores que 255 num byte.
- Usar `htons` em um lado e escrita manual no outro, o que duplica a conversão.

Checkpoint

Escrever 0x1234 produz os bytes `12 34`, e a leitura desses bytes devolve 0x1234. Teste também 0x0100 e 0x000F.

Fase 2. Codificação do nome (QNAME)

Objetivo

Transformar um texto como `unb.br` na sequência DNS: comprimento do label, bytes do label, repetido, terminado por zero.

Contrato

- Entrada: nome textual, destino, capacidade do destino.
- Saída: quantidade de bytes escritos.
- Retorno: sucesso ou erro (entrada ou capacidade inválida).
- Em caso de erro, nada deve ter sido escrito além da capacidade do destino.

O que fazer, passo a passo

1. **Copiar para um buffer limitado.** Não manipule `argv` diretamente. Se a entrada for maior que o limite razoável (255 caracteres), rejeite.
2. **Normalizar o ponto final.** Remova no máximo um ponto no fim, para que `unb.br.` seja equivalente a `unb.br`. Dois pontos finais é erro.
3. **Rejeitar entradas vazias.** String vazia, ou só ".", são inválidas para este trabalho.
4. **Percorrer os componentes manualmente.** Não use `strtok`: ele junta delimitadores consecutivos e esconde rótulos vazios como em `a..br`. Procure cada ponto à mão.

5. Validar cada label:

- comprimento entre 1 e 63;
- somente letras ASCII, dígitos e hífen;
- não começar nem terminar com hífen.

6. **Verificar a capacidade antes de cada escrita.** Escreva o byte de comprimento e depois os bytes do label.

7. **Verificar o limite total.** A forma codificada completa (comprimentos + conteúdo + zero final) deve ter no máximo 255 bytes.

8. **Escrever o zero terminal** e devolver o total escrito.

Decisões e cuidados

- Nomes internacionalizados: o programa só aceita ASCII. Se o usuário precisar de acentos, ele deve fornecer Punycode (`xn--...`). Documente isso no README.
- Letras maiúsculas e minúsculas: DNS ignora diferença. Você pode manter o texto original ao enviar, mas deve comparar sem diferenciar maiúsculas ao validar a resposta (Fase 6).
- Mantenha uma versão normalizada do nome (sem ponto final) para usar nas mensagens impressas.

Erros comuns

- Contar o ponto como parte do label.
- Esquecer o zero final ou contá-lo fora do limite de 255.
- Escrever o comprimento e descobrir depois que o conteúdo não cabe.

Tabela de testes

Entrada	Resultado esperado
<code>unb.br</code>	<code>03 75 6e 62 02 62 72 00</code> (8 bytes)
<code>unb.br.</code>	igual ao de <code>unb.br</code>
<code>a.br</code>	erro, sem escrita
texto vazio	erro
label de 63 caracteres	aceito
label de 64 caracteres	erro antes de escrever
nome codificado com 256 bytes ou mais	erro
<code>-a.br</code> ou <code>a-.br</code>	erro
espaço ou byte não ASCII	erro

Checkpoint

Todos os casos acima passam sem usar rede.

Fase 3. Montagem da mensagem de consulta

Objetivo

Produzir o payload DNS completo, que o sistema operacional embrulhará em UDP e IP.

Estrutura da consulta

Offset	Tamanho	Campo	Valor
0	2	ID	16 bits aleatórios
2	2	FLAGS	0x0100 (apenas RD, recursão desejada)
4	2	QDCOUNT	1
6	2	ANCOUNT	0
8	2	NSCOUNT	0
10	2	ARCOUNT	0
12	variável	QNAME	labels terminados em zero
depois	2	QTYPE	15 (MX)
depois	2	QCLASS	1 (IN)

O que fazer

1. Zere o buffer de 512 bytes antes de começar, para que os contadores de resposta fiquem em zero sem escrita adicional.
2. **ID aleatório:**
 - abra `/dev/urandom` em modo binário;
 - leia exatamente 2 bytes e feche o arquivo;
 - se a abertura ou a leitura falhar, imprima o erro em `stderr` e encerre;
 - leia o ID **uma vez** por consulta lógica e reutilize-o nas retransmissões.
3. Escreva ID, FLAGS e QDCOUNT com as funções da Fase 1.
4. Chame a codificação do nome no offset 12 e **verifique o retorno** antes de escrever QTYPE e QCLASS.
5. Calcule o tamanho final da consulta: 12 + tamanho do QNAME + 4.
6. Devolva esse tamanho para quem chama. No envio, use-o, **nunca** o tamanho do buffer inteiro.

Decisões e cuidados

- Para testes, deve existir uma forma de fixar o ID (por exemplo, a função de montagem recebe o ID como parâmetro, e só `main` o sorteia). Assim o teste compara os bytes exatos.
- Nunca use `rand()` com semente de tempo: o ID previsível facilita respostas falsas.

Erros comuns

- Enviar 512 bytes em vez de 24.
- Sortear o ID de novo a cada retransmissão, o que descarta respostas atrasadas da primeira tentativa.
- Esquecer de zerar o buffer.

Checkpoint

Com ID fixo 0x1234 e domínio `unb.br`, o resultado tem exatamente 24 bytes:

12 34 01 00 00 01 00 00 00 00 00 00 03 75 6e 62 02 62 72 00 00 0f 00 01

Fase 4. Endereço do servidor e socket UDP

Objetivo

Preparar o destino e o canal de comunicação.

O que fazer

1. **Converter o IP literal.** Tente primeiro IPv4 e depois IPv6 com `inet_pton`. Em ambos, a porta é 53 em ordem de rede. Se nenhuma das duas conversões retornar sucesso, o IP é inválido: erro em `stderr`.
2. **Armazenar em `sockaddr_storage`**, que comporta qualquer família. Guarde também o tamanho correto da estrutura (IPv4 e IPv6 têm tamanhos diferentes).
3. **Criar o socket** com a família correspondente e tipo datagrama. Teste o retorno.
4. **Não fazer `bind`**. O sistema escolhe uma porta de origem efêmera. Não use a porta local 53 e não execute o cliente como administrador.
5. **Garantir que o socket seja fechado** em todos os caminhos de saída (sucesso, erro, timeout).

Decisões e cuidados

- `inet_pton` apenas converte texto em binário. Ele **não** consulta DNS, então o uso é permitido.
- Defina um padrão de limpeza: um único ponto de saída em `main`, que sempre fecha o socket, evita vazamentos.

Erros comuns

- Aceitar nomes de host como servidor (o enunciado manda IP literal).
- Usar o tamanho de `sockaddr_in` para um endereço IPv6.

Checkpoint

IPs `8.8.8.8` e `2001:4860:4860::8888` são aceitos. `abc`, `999.1.1.1` e vazio produzem erro e nada é enviado.

Fase 5. Troca de mensagens: envio, espera e retransmissão

Objetivo

Garantir 3 tentativas de 2 segundos cada, sem nunca esperar indefinidamente.

Algoritmo, descrito em passos

1. Para tentativa de 1 a 3:
 1. Envie a consulta (os mesmos bytes, o mesmo servidor, o mesmo socket). Considere sucesso apenas se o número de bytes enviados for igual ao tamanho da consulta. Um erro local no envio encerra com erro.
 2. Calcule o prazo: relógio monotônico atual em ms + 2000.
 3. Enquanto o relógio for menor que o prazo:
 1. Calcule o tempo restante.
 2. Espere dados com `poll` por no máximo esse tempo.
 3. Se `poll` foi interrompido por sinal (EINTR), recalcule o restante e volte.
 4. Se esgotou o tempo, saia do laço interno e vá para a próxima tentativa.
 5. Se houve erro irreversível, encerre com erro local.
 6. Leia um datagrama com `recvfrom`, usando o buffer de 65535 bytes.

7. Confira a origem: família, endereço e porta devem ser os do servidor.
 8. Confira ID e pergunta. Se não baterem, **descarte e continue esperando**.
 9. Se for uma resposta válida para a consulta, devolva o resultado.
2. Após 3 tentativas sem resposta válida, devolva falha de coleta.

Decisões e cuidados

- **Relógio monotônico:** use `clock_gettime` com `CLOCK_MONOTONIC`. O relógio de parede pode saltar e quebrar o prazo.
- **Recalcular o restante** a cada iteração é essencial. Se um pacote alheio chegar aos 1,9 s, a espera total não pode virar 4 s.
- **Reinicializar o tamanho da estrutura de origem** antes de cada `recvfrom`, pois ele é parâmetro de entrada e saída.
- **Comparar origem campo a campo**, não com `memcmp` da estrutura inteira, que pode incluir bytes de preenchimento diferentes. Para IPv6, considere também o escopo, quando aplicável.
- **Preservar socket e ID** entre tentativas. Uma resposta atrasada à primeira tentativa continua sendo válida na segunda.
- Total sem resposta: cerca de 6 segundos. Uma pequena variação do escalonador é normal.
- NXDOMAIN e "sem MX" são respostas válidas. **Não** repetir a consulta para elas.
- Mensagens técnicas ("timeout", "pacote descartado") vão para `stderr`.

Erros comuns

- Fazer 4 envios (uma tentativa original mais 3 repetições). O total pedido é 3.
- Reiniciar o prazo a cada pacote recebido.
- Usar `recv` sem ler o endereço de origem e aceitar respostas de qualquer host.
- Bloquear em `recvfrom` sem `poll`.

Checkpoint

- Contra um servidor real, a resposta chega na primeira tentativa.
- Contra uma porta local sem resposta, o programa envia 3 datagramas e termina em cerca de 6 s.

Fase 6. Leitura de nomes comprimidos

Objetivo

Decodificar nomes que usam ponteiros de compressão, sem se perder e sem travar.

Conceito

Num nome DNS, cada byte inicial indica o que vem a seguir:

Dois bits superiores	Significado
00	tamanho de label (1 a 63), seguido do conteúdo
11	ponteiro de 2 bytes: os 14 bits restantes são o offset a partir do início da mensagem
01 ou 10	formato inválido, rejeitar
byte 0	fim do nome

Exemplo: o par **C0 0C** aponta para o byte 12, onde começa o QNAME da pergunta.

Contrato

- Entrada: mensagem, tamanho da mensagem, cursor (por referência), buffer de saída e sua capacidade.
- Saída: nome textual expandido, terminado em **\0**.
- O cursor **deve** terminar logo após a **representação original** do nome no ponto de leitura, e não no fim do nome apontado.

O ponto mais importante: duas posições

Mantenha duas variáveis diferentes:

- **posição de leitura:** muda ao seguir ponteiros.
- **posição de continuação:** guarda onde o próximo campo do registro começa. É definida **apenas na primeira vez** que se encontra um ponteiro ou o byte zero e **nunca** é alterada depois.

Se o cursor do registro for movido para dentro do sufixo apontado, todo o restante do parsing sai errado.

O que fazer

1. Comece com a posição de leitura igual ao cursor de entrada, continuação indefinida e saída vazia.
2. Repita com um limite máximo de passos (por exemplo, o tamanho da mensagem ou 128):
 1. Exija que a posição esteja dentro da mensagem.
 2. Se o byte for zero: se a continuação está indefinida, defina como posição + 1. Termine a string e atualize o cursor. Retorne sucesso.
 3. Se os dois bits superiores forem **11**: exija que exista o segundo byte, calcule o destino, exija que esteja dentro da mensagem, defina a continuação como posição + 2 (se ainda indefinida) e salte.
 4. Se os dois bits superiores forem **01** ou **10**: erro.
 5. Caso contrário é um label: exija 1 a 63 e que todo o conteúdo caiba na mensagem. Acrescente um ponto se já houver label na saída, copie o conteúdo e avance.
3. Se o limite de passos acabar sem terminar, erro de formato (ponteiro circular).
4. Valide o tamanho do nome expandido: no máximo 255 caracteres, deixando espaço para **\0**.
5. Para o nome raiz (apenas o byte zero), a saída é o texto ".".

Decisões e cuidados

- **Proteção contra loops:** um ponteiro que aponta para si mesmo, ou dois ponteiros que apontam um para o outro, **precisam** falhar rapidamente. O limite de passos é a defesa.
- **Segurança de exibição:** como o nome vem da rede, não imprima bytes de controle diretamente no terminal. Aceite apenas caracteres imprimíveis ou escape os demais.
- Pense em ponteiros só para trás? A norma permite ponteiros para qualquer posição anterior; aceitar qualquer destino dentro da mensagem e limitar passos é a abordagem mais simples e segura.

Erros comuns

- Mover o cursor do registro para o final do nome apontado.
- Esquecer de verificar **posição + 1 < n** antes de ler o segundo byte do ponteiro.
- Não limitar o tamanho da saída e estourar o buffer.

Testes sem rede

Montagem manual	Resultado esperado
unb.br no offset 12 e, em outro local, 04 "mail" C0 0C	mail.unb.br; o cursor avança 7 bytes no local original
ponteiro com destino fora da mensagem	erro, sem travar
ponteiro para si mesmo	erro, sem travar
label cujo tamanho ultrapassa a mensagem	erro
byte inicial 0x40 ou 0x80	erro
só o byte zero	."

Checkpoint

Todos os testes passam, incluindo os de falha, e nenhum causa travamento ou leitura fora do buffer.

Fase 7. Validação do cabeçalho e da pergunta

Objetivo

Garantir que o pacote recebido é mesmo a resposta da nossa consulta antes de confiar em qualquer conteúdo.

Cabeçalho da resposta (12 bytes)

Campo	Onde	Como interpretar
ID	bytes 0-1	igual ao enviado
FLAGS	bytes 2-3	veja a tabela abaixo
QDCOUNT	bytes 4-5	deve ser 1
ANCOUNT	bytes 6-7	quantidade de registros na seção Answer
NSCOUNT	bytes 8-9	quantidade na seção Authority
ARCOUNT	bytes 10-11	quantidade na seção Additional

Bits de FLAGS que interessam

Máscara	Nome	Esperado ou uso
0x8000	QR	deve ser 1 (resposta)
0x7800	OPCODE	deve ser 0
0x0200	TC	se 1, resposta truncada
0x000F	RCODE	0 ok; 3 nome inexistente; demais, erro

O que fazer

1. Exija tamanho recebido de pelo menos 12 bytes.
2. Compare o ID. Se diferente, o pacote é **irrelevante** (ignorar, não é erro).
3. Exija QR = 1, OPCODE = 0 e QDCOUNT = 1. Caso contrário, ignore.
4. Decodifique o nome da pergunta com a função da Fase 6 e confira que cabem 4 bytes depois dele.
5. Compare o nome, **sem diferenciar maiúsculas de minúsculas**, com o domínio consultado. Compare também tipo MX e classe IN.

6. Se qualquer item de identidade falhar, o pacote é descartado e a espera continua dentro do prazo original.
7. Depois da identidade confirmada, veja o TC. Se TC = 1, **não** extraia resultado parcial: informe falha de coleta. Como o trabalho exige UDP, não há fallback TCP, e isso deve constar no README.
8. Veja o RCODE: 3 indica domínio inexistente. Qualquer outro valor diferente de 0, como SERVFAIL (2) ou REFUSED (5), é falha de coleta, **não** ausência de MX.

Decisões e cuidados

- Pacotes inválidos ou alheios **não consomem uma tentativa e não reiniciam o prazo**.
- Diferencie "irrelevante" (ignorar e continuar) de "relevante com erro" (encerrar tentativas com falha de coleta).

Checkpoint

Testes com buffers fabricados: ID diferente é ignorado; TC = 1 vira falha de coleta; RCODE 3 vira NXDOMAIN; RCODE 2 e 5 viram falha de coleta.

Fase 8. Percorrer os registros de recurso

Objetivo

Extrair cada registro (RR) das seções Answer, Authority e Additional com segurança.

Formato de um RR

Nome (comprimido ou não), seguido de 10 bytes fixos e dos dados:

Parte	Tamanho	Observação
NAME	variável	use a leitura da Fase 6
TYPE	2	MX = 15, CNAME = 5, SOA = 6, NS = 2
CLASS	2	IN = 1
TTL	4	lido para avançar, não impresso
RDLLENGTH	2	tamanho do RDATA
RDATA	RDLLENGTH	conteúdo depende do tipo

O que fazer

1. Comece o cursor logo após a pergunta (cabeçalho + QNAME + 4).
2. Para cada RR de Answer, na ordem do ANCOUNT:
 1. Leia o nome do dono (owner) e avance o cursor.
 2. Exija 10 bytes disponíveis.
 3. Leia tipo, classe e RDLLENGTH. Calcule início e fim do RDATA.
 4. Exija que o fim esteja dentro da mensagem.
 5. Se for tipo MX e classe IN: exija RDLLENGTH de pelo menos 3 (2 de preferência mais pelo menos 1 do nome). Leia a preferência e depois o nome do servidor com a leitura da Fase 6, numa variável de cursor temporária. Exija que o cursor temporário termine **exatamente** em fim.
 6. Se for CNAME: leia o nome de destino no RDATA, também com cursor temporário, e guarde o par (owner, destino).
 7. Guarde o resultado.

8. Mova o cursor do registro para o fim do RDATA, independentemente do tipo.
3. Percorra Authority e Additional da mesma forma, para posicionar corretamente o fim da mensagem e para registrar SOA e NS da seção Authority, usados na classificação de ausência de MX.
4. Antes de qualquer saída, guarde tudo em estruturas na memória. Uma falha de formato em um registro posterior não pode deixar saída parcial.

Decisões e cuidados

- Para tipos desconhecidos, apenas pule o RDATA usando RDLENGTH.
- Conteúdo do RDATA de nomes também pode usar compressão. Por isso a leitura de nomes recebe a mensagem inteira, não só o RDATA.
- Defina um limite razoável de MX armazenados (por exemplo, 32 ou mais); ao exceder, ignore o excedente com aviso em `stderr` ou use alocação dinâmica com liberação correta.
- Mantenha a preferência em memória mesmo sem imprimi-la. Ela permite ordenar se desejado.

Erros comuns

- Confiar em RDLENGTH sem checar o limite da mensagem.
- Esquecer que o nome do servidor no MX pode ser ponteiro.
- Usar o mesmo cursor para o RDATA e para o registro, perdendo a posição.

Checkpoint

Com uma resposta fabricada contendo dois MX com nome comprimido, o parser devolve os dois, com preferências e servidores corretos, e o cursor termina exatamente no fim da seção.

Fase 9. Classificação do resultado

Objetivo

Transformar o que foi lido em uma das quatro situações finais.

Tabela de decisão

Situação validada	Classificação	Saída
RCODE = 3	domínio inexistente	<code>Domínio X nao encontrado</code>
RCODE = 0 e há MX relevante	sucesso	<code>uma linha X <> servidor por MX</code>
RCODE = 0, sem MX relevante, com SOA em Authority e sem referral	sem MX	<code>Domínio X nao possui entrada MX</code>
Timeout após 3 envios, TC = 1, SERVFAIL, REFUSED, cadeia incompleta, resposta vazia sem evidência	falha de coleta	<code>Nao foi possível coletar entrada MX para X</code>

Confira no enunciado a grafia exata das mensagens (acentos incluídos) e copie-a literalmente.

Tratamento de CNAME

1. Comece com o nome consultado como nome atual.
2. Procure em Answer um CNAME cujo owner seja o nome atual. Se encontrar, o destino passa a ser o nome atual.
3. Repita no máximo 8 vezes. Guarde os nomes visitados e, se algum se repetir, trate como erro de formato.

4. Aceite apenas MX cujo owner seja o **nome final** da cadeia.
5. Se houver CNAME mas nenhum MX final nem SOA que prove ausência, é falha de coleta. Não abra consultas extras.
6. Na impressão de sucesso, o primeiro campo da linha continua sendo o **domínio original consultado**, conforme o formato do enunciado.

Regra de "sem MX"

- RCODE 0, nenhum MX relevante e um SOA em Authority.
- Referral (apenas NS na Authority, sem SOA) não prova ausência. É falha de coleta.
- Respostas vazias sem SOA ficam como falha de coleta. É conservador, mas nunca confunde delegação com ausência.

Null MX

- Um MX com preferência 0 e servidor "." significa "este domínio não recebe e-mail" (RFC 7505).
- Imprima como linha normal: `X <> ..` Não trate como ausência de MX.

Múltiplos MX

- Imprima todos os relevantes, na ordem recebida. Ordenar por preferência é opcional.
- Não adicione preferência, TTL nem texto explicativo às linhas de sucesso.

Separação de saídas e código de retorno

- Resultados e as quatro mensagens exigidas vão para `stdout`.
- Detalhes técnicos ("resposta truncada", "IP inválido", "timeout") vão para `stderr`.
- Convenção sugerida e documentada no README: 0 quando há MX obtido, 1 para qualquer outro caso. Isso é uma escolha do guia, não do enunciado.

Checkpoint

Para cada linha da tabela de decisão, uma resposta fabricada produz exatamente a saída esperada.

Parte C. Testes

C.1 Estratégia em dois níveis

1. **Unitários com bytes conhecidos:** utilitários de 16 bits, codificação de nome, montagem de consulta, leitura de nomes, validação e classificação. Não usam rede e são totalmente determinísticos.
2. **Integração:** o programa completo contra servidores reais e contra um servidor local simulado.

Sugestão prática: coloque os testes unitários em um segundo arquivo `.c` fora do pacote de entrega, ou ative-os por uma opção de compilação, para não alterar o programa entregue.

C.2 Matriz de casos

Caso	Evidência de aprovação
consulta de <code>unb.br</code> com ID fixo	exatamente os 24 bytes esperados
MX com nome comprimido	nome completo correto e cursor correto
dois ou mais MX	uma linha por registro relevante
NXDOMAIN sintético	mensagem de domínio não encontrado
NOERROR sem MX com SOA	mensagem de ausência de MX
CNAME seguido de MX do destino	resultado ligado ao domínio original
CNAME com laço	falha de coleta, sem travar
Null MX	linha <code>X <> .</code>
TC, SERVFAIL, REFUSED	falha de coleta
ponteiro circular, RDLENGTH inválido	sem travar e sem ler fora do buffer
pacote com ID ou origem diferentes	ignorado, sem reiniciar o prazo
servidor silencioso	3 consultas e espera perto de 6 s
IP inválido, domínio inválido	erro em <code>stderr</code> , nenhum pacote enviado

C.3 Testes de integração com servidores reais

- Executar com um domínio com MX conhecido, um domínio inexistente e um subdomínio sem MX, contra resolvedores públicos (por exemplo `8.8.8.8` e `1.1.1.1`).
- Opcionalmente comparar com `dig @servidor domínio MX +noall +answer`, **apenas como ferramenta de verificação**.
- Compare o **conjunto** de servidores, não a ordem das linhas.
- Os domínios do enunciado são apenas exemplos. Registros reais mudam, e um resultado diferente do exemplo não prova erro.
- Se a rede bloquear UDP 53 externo, use um resolvedor acessível na rede local e registre essa condição.

C.4 Teste de timeout controlado

1. Em ambiente isolado, abra um servidor UDP local que escute a porta de teste e apenas descarte e conte os datagramas.
2. Aponte o cliente para ele. O cliente fixa a porta 53 por exigência, então esse teste pode precisar de privilégio para o servidor de teste, ou de um redirecionamento de porta. O cliente em si não exige

privilegio.

3. Meça: 3 datagramas recebidos e cerca de 6 segundos de duração.
4. Não confie em um IP "silencioso" da internet; a rede pode devolver erro imediato.

C.5 Testes de robustez

- Compile uma vez com sanitizadores (`-fsanitize=address,undefined`) e rode todos os testes unitários. Eles encontram leitura fora do limite.
- Teste respostas truncadas em cada posição possível (cortar o pacote byte a byte). O parser nunca pode falhar de forma insegura.
- Teste com `valgrind` em Linux, se disponível.

Parte D. Documentação (20% da nota)

D.1 Princípios

- Documente o programa **realmente entregue**. Este guia orienta a construção, mas não substitui o registro do ambiente, dos testes e das limitações finais.
- Nunca apresente exemplos deste material como se fossem testes que você executou.
- Mostre o comando, a saída real e uma frase explicando o resultado.

D.2 Estrutura do README .md

1. **Autores:** nome completo e matrícula de cada integrante.
2. **Sistema operacional:** distribuição e versão em que foi testado.
3. **Ambiente de desenvolvimento:** editor, versão do compilador e ferramentas realmente usadas.
4. **Como construir:** linha de compilação completa. Se houver mais de um `.c`, atualizar o comando.
5. **Como executar:** formato `./meu_cliente DOMINIO IP_DO_SERVIDOR`, com exemplos.
6. **Instruções de uso e telas:** transcrições reais do terminal para cada situação (sucesso, NXDOMAIN, ausência de MX, falha de coleta).
7. **Limitações conhecidas:** apenas MX/IN, UDP na porta 53, sem fallback TCP, sem EDNS, sem DNSSEC, sem cache, nomes ASCII/Punycode, servidor por IP literal, até oito aliases, cadeia incompleta é inconclusiva, aleatoriedade via `/dev/urandom` (Linux), sem suporte a Windows nativo. Declare se for apenas IPv4.
8. **Testes realizados:** casos, ambiente e resultados observados.
9. **Convenção de código de retorno** e separação entre `stdout` e `stderr`.

D.3 Documentação dentro do código

- Um comentário curto no topo de cada função: o que faz, parâmetros, retorno e pré-condições (especialmente sobre limites).
- Comente decisões não óbvias (por exemplo, por que há duas posições na leitura de nomes).
- Remova logs de depuração do `stdout` antes de entregar.

Parte E. Revisão e entrega

E.1 Checklist funcional

- ☐ Argumentos na ordem domínio, IP
- ☐ Payload montado manualmente, sem biblioteca DNS
- ☐ ID aleatório de 2 bytes lido de `/dev/urandom`
- ☐ FLAGS 0x0100 e exatamente uma pergunta
- ☐ Contadores de resposta zerados na consulta
- ☐ UDP, porta 53, sem `bind`
- ☐ 3 tentativas de 2 s, relógio monotônico
- ☐ Respostas alheias ignoradas sem reiniciar o prazo
- ☐ Nomes comprimidos lidos corretamente
- ☐ TC, SERVFAIL, REFUSED tratados como falha de coleta
- ☐ CNAME seguido com limite e detecção de repetição
- ☐ Null MX tratado
- ☐ Mensagens exatas em `stdout`, detalhes em `stderr`
- ☐ Socket fechado em todos os caminhos

E.2 Checklist de qualidade

- ☐ Compila sem avisos relevantes em Linux
- ☐ Todo acesso a buffer é precedido de verificação de limite
- ☐ Testes com respostas sintéticas e com servidor real
- ☐ Sem logs de depuração no `stdout`
- ☐ README preenchido com dados reais

E.3 Empacotamento

1. Inclua apenas código-fonte e documentação (`meu_cliente.c`, `README.md` e outros `.c/.h`, se houver).
2. Nome do ZIP conforme o enunciado:
`nome_sobrenome_matricula_nome_sobrenome_matricula_trab01.zip`.
3. **Não** inclua executáveis, arquivos `.o` nem binários de teste.
4. Extraia o ZIP numa pasta limpa, compile com o comando do README e rode um teste antes de enviar.
5. Envie pelo Sigaa. O enunciado não informa data-limite; confirme no Moodle.

Parte F. Ordem sugerida de trabalho para a dupla

Ordem	Tarefa	Quem pode fazer em paralelo
1	Fases 0 e 1 (ambiente, utilitários)	ambos
2	Fase 2 e 3 (nome e consulta)	peessoa A
2	Fase 6 (nomes comprimidos) com testes manuais	peessoa B
3	Fases 4 e 5 (socket, prazos)	peessoa A
3	Fases 7 e 8 (validação e registros)	peessoa B
4	Fase 9 (classificação e saída)	ambos
5	Testes, README e revisão cruzada	ambos

Parte G. Referências

- RFC 1035, formato DNS e MX, seções 3.3.9 e 4.1.1 a 4.1.4. <https://www.rfc-editor.org/rfc/rfc1035>
- RFC 1034, conceitos de resolução. <https://www.rfc-editor.org/rfc/rfc1034>
- RFC 7505, Null MX. <https://www.rfc-editor.org/rfc/rfc7505.html>
- `poll(2)`: <https://man7.org/linux/man-pages/man2/poll.2.html>
- `inet_pton(3)`: https://man7.org/linux/man-pages/man3/inet_pton.3.html
- `recvfrom(2)`: <https://man7.org/linux/man-pages/man2/recvfrom.2.html>
- Enunciado do trabalho, Prof. Tiago Alves.